Week 7
Policy Gradients and REINFORCE

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives 3

1 Motivation and intuition 4

2 Mathematical formulation 5
2.1 Setting and notation . . . 5
2.2 The Policy Gradient Theorem . . . 5
2.3 Variance reduction: baselines . . . 6
2.4 Score function vs. reparameterisation . . . 7
2.5 Empirical-variance plot setup . . . 7

3 Algorithms 9
3.1 Worked numerical example . . . 9

4 Implementation walkthrough 10
4.1 Categorical policy on CartPole . . . 10
4.2 One gradient update . . . 10

5 Empirical results 11

6 Safety angle 13

7 Matching application 14

8 Exercises 15

9 Further reading 16

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this week you will be able to:

1. State and *derive* the policy gradient theorem from first principles, including the log-derivative (REINFORCE) trick.
2. Distinguish the **score-function** estimator from the **reparameterisation** (pathwise) estimator and explain why one is unbiased for discrete actions while the other gives lower variance for continuous ones.
3. Implement REINFORCE with a learned value baseline in $\sim$80 lines of PyTorch and demonstrate the variance-reduction effect empirically on CartPole-v1.
4. Explain why subtracting any state-dependent function $b(s)$ preserves unbiasedness and how to derive the optimal baseline.
5. Translate the algorithm to a *ranking* setting: REINFORCE for top-K list selection in the YouTube paper style.
6. Identify three classical failure modes of vanilla PG and prescribe a mitigation for each.

1 Motivation and intuition

Value-based methods (DQN family, weeks 5–6) parametrise $Q_\theta(s, a)$ and act greedily. The catch: $\arg\max_a$ does not differentiate, the actor is implicit, and continuous action spaces force awkward workarounds (NAF, DDPG). **Policy-gradient methods** flip the design: parametrise the actor $\pi_\theta(a \mid s)$ directly, define the objective

$$J(\theta) \;=\; \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{T} \gamma^t r_t \right],$$

and climb its gradient.

This trade-off matters most when the optimal policy is stochastic — e.g. in partially observed games or in matching problems where exploring a Bernoulli draw is the right hedge.

Intuition. Think of θ as the weights of a recommender, and $\pi_\theta(a \mid s)$ as the probability of suggesting candidate a for context s. The policy gradient says: *nudge θ in whichever direction makes the actions that were followed by high reward more likely, and the ones followed by low reward less likely.* That sentence is the entire algorithm.

Anchoring to supervised learning

If we had a dataset $(s_i, a_i^\star)$ of expert decisions we would minimise $-\sum_i \log \pi_\theta(a_i^\star \mid s_i)$ — ordinary cross-entropy. REINFORCE replaces the binary label *expert* with the continuous label *advantage*: good actions get up-weighted log-likelihood, bad actions get down-weighted. The structural similarity to maximum-likelihood is why PG fits seamlessly inside deep-learning pipelines.

This is exactly imitation learning / behavioural cloning; it is the limit of PG when the advantage is replaced by $+1$ on expert actions.

Why we will not abandon value methods

The exact on-policy score-function estimator is unbiased under its stated assumptions, but can have high variance. Practical methods introduce critics, normalisation, clipping, or different objectives; these can add bias. The next four weeks examine those trade-offs rather than preserving unbiasedness automatically.

Common pitfall

A loss such as `-(return * log_prob).mean()` can be correct for an appropriate objective and sampling scheme. A declining learning curve is not by itself proof of estimator variance: also inspect the gradient sign, return indexing, optimiser scale, exploration, and implementation errors.

2 Mathematical formulation

2.1 Setting and notation

We work in a discounted MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ with stochastic policy $\pi_\theta : \mathcal{S} \to \Delta(\mathcal{A})$, trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, \ldots, s_T)$, return $G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t$, and trajectory distribution

$$p_\theta(\tau) \;=\; \rho_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t)\, p(s_{t+1} \mid s_t, a_t).$$

We seek $\theta^\star = \arg\max_\theta J(\theta)$ where $J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G(\tau)]$.

Definition 2.1.1 (Log-derivative / score-function identity)**.** For any parametric density $p_\theta(x)$ with support not depending on θ,

$$\nabla_\theta p_\theta(x) \;=\; p_\theta(x)\, \nabla_\theta \log p_\theta(x).$$

This single algebraic identity is the engine of REINFORCE. The hypothesis "support does not depend on θ" is why deterministic policies need a different trick (reparameterisation, week 10).

2.2 The Policy Gradient Theorem

Theorem 2.2.1 (Policy Gradient Theorem, Sutton et al. 1999)**.** *For a differentiable parametric policy π_θ in a discounted MDP, the gradient of $J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G(\tau)]$ is*

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, Q^{\pi_\theta}(s_t, a_t) \right].$$

Full derivation. Start from the trajectory-level expectation:

$$J(\theta) = \int p_\theta(\tau)\, G(\tau)\, d\tau.$$

Differentiate under the integral (regularity assumed):

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau)\, G(\tau)\, d\tau.$$

Apply Definition 2.1.1:

$$\nabla_\theta J(\theta) = \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, G(\tau)\, d\tau = \mathbb{E}_{\tau \sim p_\theta}[\nabla_\theta \log p_\theta(\tau)\, G(\tau)]\,.$$

The log-likelihood of a trajectory factors:

$$\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_{t=0}^{T-1} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T-1} \log p(s_{t+1} \mid s_t, a_t).$$

The first and third terms have zero θ-gradient (the dynamics do not depend on θ), so

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Therefore

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau\left[\Big(\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big)\Big(\sum_{t'} \gamma^{t'} r_{t'}\Big)\right]. \tag{$\star$}$$

Causality trick. Because actions at time t cannot influence rewards before t, the cross-terms with $t' < t$ have zero expectation. Formally, let H_t be the full history before selecting a_t and condition on H_t, so a past reward is measurable:

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot r_{t'}] = \mathbb{E}[r_{t'} \cdot \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \mid H_t]] = 0$$

for $t' < t$, where we use $\mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)] = \nabla_\theta \int \pi_\theta(a_t \mid s_t)\, da_t = \nabla_\theta 1 = 0$. The same identity also underlies the baseline result below.

Substituting back into ($\star$):

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{t' \geq t} \gamma^{t'} r_{t'}\right].$$

Define the return-to-go $G_t \triangleq \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$, so the preceding inner sum is $\gamma^t G_t$. The outer γ^t therefore remains in the trajectory-sum gradient. For a finite horizon, include time in the state so the continuation values are well defined. One more conditioning step — the tower property over future randomness given (s_t, a_t) — replaces $\sum_{t' \geq t} \gamma^{t'-t} r_{t'}$ with $Q^{\pi_\theta}(s_t, a_t)$, since by definition

$$Q^\pi(s, a) = \mathbb{E}\left[\sum_{k \geq 0} \gamma^k r_{t+k} \,\middle|\, s_t = s, a_t = a, \pi\right].$$

Substituting this conditional expectation while retaining the outer γ^t factor gives the claim. □

Common pitfall

Many implementations drop the leading γ^t in $\gamma^t Q(s_t, a_t)$ — this introduces a small bias but works better empirically. Be aware that you are then optimising the *undiscounted average* of discounted returns, not a true policy-gradient estimator. (Thomas 2014, *Bias in Natural Actor-Critic Algorithms*, discusses this in depth.)

2.3 Variance reduction: baselines

Theorem 2.3.1 (Baseline lemma)**.** *For any function $b : \mathcal{S} \to \mathbb{R}$ (not depending on a_t),*

$$\mathbb{E}_\tau\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, b(s_t)\right] = 0.$$

Proof. Condition on s_t and integrate over a_t first:

$$\mathbb{E}_{a_t \sim \pi_\theta(\cdot \mid s_t)}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)] \;=\; \nabla_\theta \int \pi_\theta(a_t \mid s_t)\, da_t \;=\; \nabla_\theta 1 \;=\; 0.$$

$b(s_t)$ comes out of the inner expectation because it is independent of a_t. □

Hence we may replace $Q^\pi(s_t, a_t)$ by the **advantage**

$$A^\pi(s_t, a_t) \triangleq Q^\pi(s_t, a_t) - V^\pi(s_t)$$

without changing the gradient *in expectation*. A suitable baseline can reduce variance, but centring the return alone does not guarantee variance reduction: squared score norms also matter.

Optimal baseline. Minimising $\mathrm{Var}[\nabla_\theta \log \pi \cdot (G - b)]$ in b gives, for minimising the sum of component variances,

$$b^\star = \frac{\mathbb{E}[\|\nabla_\theta \log \pi\|^2 \, G]}{\mathbb{E}[\|\nabla_\theta \log \pi\|^2]}.$$

In practice we use $b(s) = V_\phi(s)$ trained by regression on the observed returns — a useful, cheap surrogate that need not be variance-optimal (Greensmith, Bartlett, Baxter 2004 give the formal sense in which this approximation is sound).

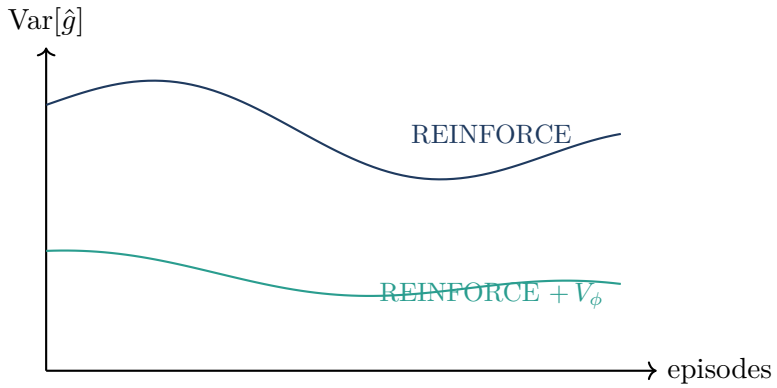


Figure 2.1: Illustrative schematic of a possible variance reduction; these curves are teaching fixtures, not measured parameter-gradient variances. The project plot in Figure 5.2 measures scalar loss-term variability.

2.4 Score function vs. reparameterisation

The estimator we just derived is the *score-function* (a.k.a. REINFORCE, likelihood-ratio) estimator. An alternative exists when the random variable can be written as a deterministic function of a parameter-free noise source:

$$a = f_\theta(\epsilon, s), \quad \epsilon \sim p(\epsilon).$$

Then $\nabla_\theta \mathbb{E}_\epsilon[\, \ell(f_\theta(\epsilon, s))\,] = \mathbb{E}_\epsilon[\nabla_\theta \ell \cdot \nabla_\theta f_\theta]$ — the **reparameterisation trick**.

Remark. For a Gaussian policy $a \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$ we can write $a = \mu_\theta(s) + \sigma_\theta(s)\,\epsilon$ with $\epsilon \sim \mathcal{N}(0, 1)$. The pathwise estimator additionally needs a differentiable downstream objective. It often has lower variance, but there is no universal ordering for arbitrary objectives and distributions. Week 10 (SAC) uses this aggressively. *Discrete* actions admit no such trick (Gumbel-softmax is a continuous relaxation, not a true reparameterisation).

	Score function (REINFORCE)	Reparameterisation
Applicability	any sampler	needs $a = f_\theta(\epsilon)$
Bias	unbiased	unbiased
Variance	high	low (deterministic gradient path)
Gradient through	$\log \pi_\theta$	π_θ itself
Used by	REINFORCE, A2C, TRPO, PPO, GRPO	SAC, TD3 actor loss, VAEs

Table 2.1: The two pillars of stochastic-gradient estimation that underpin modern RL.

2.5 Empirical-variance plot setup

The mini-project's `estimator_variance` measures the empirical variance of scalar loss contributions $\log \pi(a_t \mid s_t)(G_t - V(s_t))$ within a rollout. This is a diagnostic proxy, not the

variance of the policy-gradient estimator. A true gradient-variance measurement would use per-sample parameter gradients $\hat{g}_i = \nabla_\theta \log \pi(a_i \mid s_i)\hat{A}_i$ and compute $N^{-1}\sum_i \|\hat{g}_i\|^2 - \|\bar{g}\|^2$ under a specified sampling scheme.

3 Algorithms

REINFORCE with learned baseline (Williams 1992 + V-baseline)

Input: differentiable policy π_θ, value head V_ϕ, learning rates η_θ, η_ϕ, discount γ.
Repeat:

1. Roll out one full episode $\tau = (s_0, a_0, r_0, \ldots, s_T)$ under π_θ.
2. Compute returns-to-go $G_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$.
3. Compute advantages $A_t = G_t - V_\phi(s_t)$.
4. Standardise: $A_t \leftarrow (A_t - \bar{A})/(\hat{\sigma}_A + 10^{-8})$.
5. Policy loss: $L_\pi = -\frac{1}{T+1} \sum_t \log \pi_\theta(a_t \mid s_t)\, A_t$.
6. Value loss: $L_V = \frac{1}{T+1} \sum_t (V_\phi(s_t) - G_t)^2$.
7. $\theta \leftarrow \theta - \eta_\theta \nabla L_\pi$; $\quad \phi \leftarrow \phi - \eta_\phi \nabla L_V$.

Estimator convention: the displayed practical loss omits the outer γ^t of the exact start-state discounted gradient and standardises advantages using the same episode. These are estimator modifications; the unbiasedness theorem does not directly apply to this implementation.
Complexity per update: $\mathcal{O}(T \cdot |\theta|)$ for the policy and $\mathcal{O}(T \cdot |\phi|)$ for the value head, no replay.

3.1 Worked numerical example

Single-state, two-action MDP: $\mathcal{S} = \{s_0\}$, $\mathcal{A} = \{0, 1\}$, deterministic rewards $r(0) = 0, r(1) = 1$, episode length 1. Policy: $\pi_\theta(a = 1) = \sigma(\theta)$ (logistic).

- $J(\theta) = \sigma(\theta)$.
- $\nabla_\theta J = \sigma(\theta)(1 - \sigma(\theta))$ — the analytic ground truth.
- REINFORCE estimator with one sample: $\hat{g} = r \cdot \nabla_\theta \log \pi_\theta(a)$. If $a = 1$ (prob σ) we get $1 \cdot (1 - \sigma)$; if $a = 0$ (prob $1 - \sigma$) we get $0 \cdot (-\sigma) = 0$.
- Expectation: $\sigma(1 - \sigma) + (1 - \sigma) \cdot 0 = \sigma(1 - \sigma)$. The estimator is exact in expectation; the variance is $\sigma(1 - \sigma)^3$. It tends to zero as $\sigma \to 0$, but variance relative to the squared mean diverges.

Subtract baseline $b = V^\pi = \sigma$; advantage $A = r - \sigma$, $A = 1 - \sigma$ on $a = 1$ and $A = -\sigma$ on $a = 0$. Both branches now contribute non-zero gradients, but this baseline is not generally variance-minimising. Here the optimal scalar baseline is $b^* = 1 - \sigma$, giving the same gradient $\sigma(1 - \sigma)$ on both branches. A value baseline can even increase variance; see the teaching guide.

4 Implementation walkthrough

4.1 Categorical policy on CartPole

```python
class CategoricalPolicy(nn.Module):
    def __init__(self, obs_dim, n_actions, hidden=(64, 64)):
        super().__init__()
        from common.networks import mlp
        self.net = mlp([obs_dim, *hidden, n_actions])

    def forward(self, obs):
        return torch.distributions.Categorical(logits=self.net(obs))

    def sample(self, obs):
        dist = self(obs)
        a = dist.sample()
        return a, dist.log_prob(a), dist.entropy()
```

4.2 One gradient update

```python
def update(traj, policy, value, opt_pi, opt_v, gamma=0.99):
    obs, acts, rews = traj
    returns = discount_cumsum(rews, gamma)
    returns = torch.as_tensor(returns, dtype=torch.float32)

    with torch.no_grad():
        v = value(obs).squeeze(-1)
    adv = returns - v
    adv = (adv - adv.mean()) / (adv.std() + 1e-8)

    dist = policy(obs)
    logp = dist.log_prob(acts)
    pi_loss = -(logp * adv).mean()
    v_loss = F.mse_loss(value(obs).squeeze(-1), returns)

    opt_pi.zero_grad(); pi_loss.backward(); opt_pi.step()
    opt_v.zero_grad();  v_loss.backward();  opt_v.step()
    return pi_loss.item(), v_loss.item()
```

The full trainer lives in `project/train.py`; see also `project/agent.py` for the `REINFORCEAgent` class consumed by the workbook.

5 Empirical results

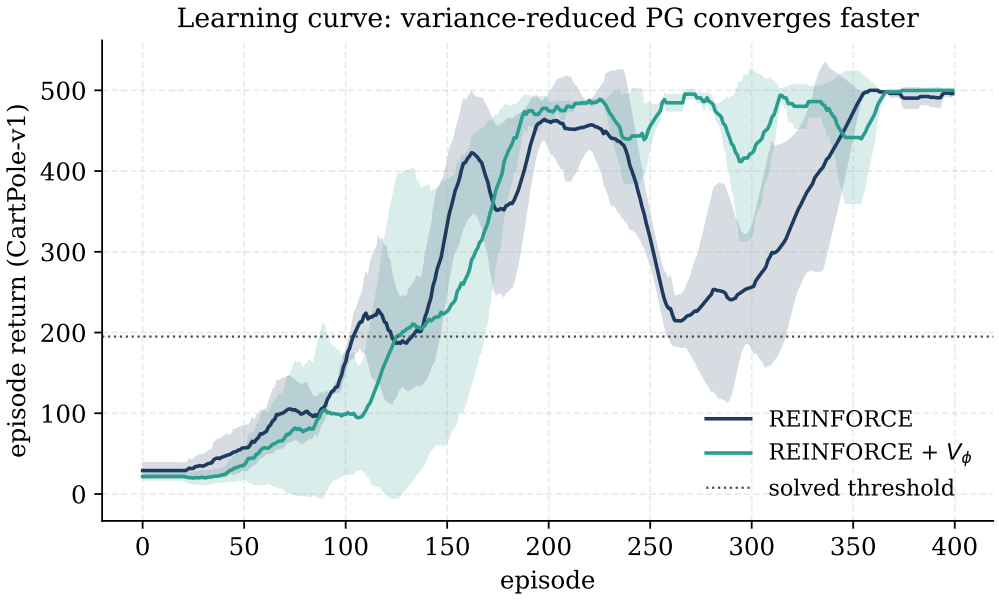


Figure 5.1: CartPole-v1 returns averaged across three seeds. REINFORCE+baseline (teal) reaches return $\geq$ 195 within $\sim$200 episodes; vanilla REINFORCE (blue) trails by a wide margin and exhibits a noisier curve.

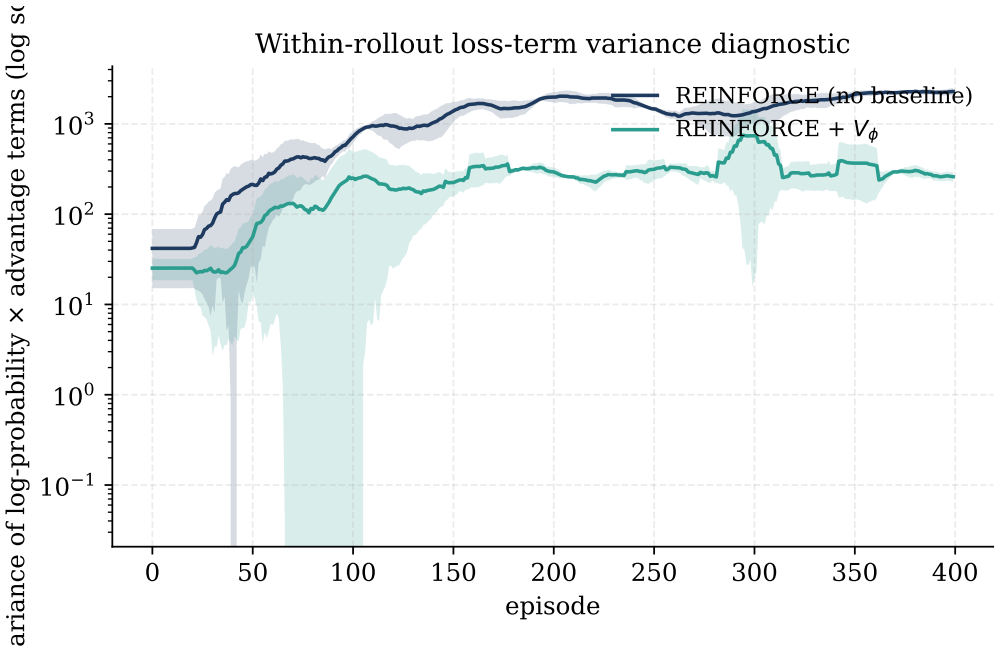


Figure 5.2: Within-rollout variance of scalar log-probability times advantage terms, with and without a learned V_ϕ baseline. This diagnostic is not the variance of parameter gradients and establishes no universal reduction factor.

6 Safety angle

Safety angle

REINFORCE in production — be it a router for high-stakes leads or an LLM fine-tuner — has three failure modes worth surfacing now, because every later policy method inherits them.

1. **Premature entropy collapse.** A few lucky early returns can push the policy to a near-deterministic, suboptimal mode in ∼50 updates. Mitigation: an explicit entropy bonus $-\beta\mathbb{E}[\log \pi(a \mid s)]$ with $\beta \approx 0.01$; or KL-trust-region (TRPO/PPO, weeks 8–9). Connects to the alignment literature on mode collapse in RLHF; see Gao, Schulman, Hilton 2022 (arXiv:2210.10760).

2. **Reward hacking.** If the surrogate reward differs from the true objective, PG will gleefully optimise the proxy. Skalse et al. NeurIPS 2024 give a clean definition of "unhackable" proxies. For matching: r = click is not r = booking; if you train on the proxy you will sell clickbait listings.

3. **Catastrophic gradients on high-magnitude rewards.** A single 1000-return outlier can shift θ off-distribution. Mitigation: advantage standardisation, gradient clipping at $\|g\|_2 \leq 0.5$. Without it, the agent can transition from learning to broken in a single mini-batch.

None of these are theoretical. In synthetic insurance's matching context, premature entropy collapse means we stop showing newer brokers entirely after a brief lucky run; reward hacking means we optimise CTR rather than NPS; gradient explosions corrupt models silently and only surface at deploy time.

7 Matching application

Matching application

REINFORCE for top-K list ranking (the YouTube paper-style application). Setup: a context s (user features, partial session), a candidate pool $\mathcal{C}(s) \subset \mathcal{V}$ (videos / brokers / candidates), a slate $A \subset \mathcal{C}(s)$ with $|A| = K$, and a reward r realised after presentation (watch-time / accepted-lead / hire). The naive softmax-over-pool gradient is

$$\nabla_\theta \mathbb{E}_{a \sim \pi_\theta(\cdot \mid s)}[r(s,a)] = \mathbb{E}_a[r(s,a) \nabla_\theta \log \pi_\theta(a \mid s)].$$

Top-K off-policy correction (Chen et al., WSDM 2019, *Top-K Off-Policy Correction for a REINFORCE Recommender System*): sampling K items without replacement, the per-item gradient picks up a multiplier $\lambda_K(s,a) = K(1 - \pi_\theta(a \mid s))^{K-1}$ — this is the correction that turned PG-for-recsys from a research idea into the YouTube recommender.

Concrete translation to synthetic ATS / synthetic insurance.

- **State s:** candidate profile (or lead profile) embedding + open-role / open-broker pool snapshot.
- **Policy $\pi_\theta(a \mid s)$:** softmax over $\mathcal{C}(s)$, parameterised by a two-tower model (candidate tower $\times$ role tower).
- **Reward r:** delayed binary — did the role get filled / did the lead convert?
- **Baseline V_ϕ:** a regression head on the same towers, fit by Monte-Carlo across the day's slates.

This is *the* first algorithm in the syllabus that you would actually deploy to drive matching. Subsequent weeks bolt on stability tricks; the kernel is REINFORCE.

8 Exercises

The graded workbook (`workbook.ipynb`) contains the full statements; below is a summary.

E1 ($\star$) Derive the log-derivative identity from scratch and verify it on $p_\theta(x) = \mathcal{N}(x; \theta, 1)$.

E2 ($\star$) Show $\mathbb{E}_a[\nabla_\theta \log \pi_\theta(a \mid s)] = 0$.

E3 ($\star\star$) Implement `discount_cumsum` in NumPy.

E4 ($\star\star$) REINFORCE on a two-armed bandit; reach optimal arm prob ≥ 0.95 in 200 episodes.

E5 ($\star\star$) Add a learned V_ϕ baseline; plot the variance with/without.

E6 ($\star\star$) Train REINFORCE+baseline on CartPole-v1 to mean return ≥ 195.

E7 ($\star\star\star$) Top-K off-policy correction on a synthetic recsys env.

E8 ($\star\star\star$, meaningful objective) SNIPS off-policy estimation for a synthetic insurance-style ε-greedy lead router.

9 Further reading

See `extended_reading.pdf`. Five papers covered: Williams 1992 (REINFORCE), Sutton et al. 1999 (PG theorem), Greensmith, Bartlett, Baxter 2004 (variance bounds), Chen et al. WSDM 2019 (top-K off-policy correction at YouTube), and Gao, Schulman, Hilton 2022 (proxy-reward over-optimisation). Sourced from `research/deeprl_core_sota.md` §2.1–2.2 and `research/bandits_matching_sota.md`.